OPENCYBERPATH

Common Attack Patterns

Recognise the pattern • understand the evidence • learn the defence

NOTEBOOK 02 • BEGINNER → INTERMEDIATE → ADVANCED • DEFENSIVE STUDY EDITION

> NOTEBOOK PROMISE Plain language first. Define every important word. Show the clue, explain what it does and does not prove, then choose a safe next step.

By the end, you can…

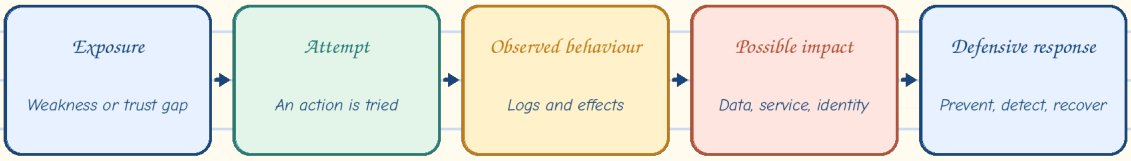
- Explain common attack types in everyday language.
- Recognise the difference between a vulnerability, a technique, and an attack outcome.
- Study fictional examples and identify useful defensive evidence.
- Connect observed behaviour to ATT&CK and OWASP references carefully.
- Practise writing safe findings without attacking real systems.

> SAFETY + HONESTY This is a defensive reference, not a guide to break into systems. All examples are fictional and non-operational. Hands-on activity belongs only in your own isolated lab or an expressly authorised training environment.

How to study: read one section, cover it, explain it aloud, then try the worked example before checking the answer.

First: what do people mean by "attack technique"?

A high-level attack story (no attack recipe)



The chain is a way to organise questions, not a promise that every incident follows the same order.

FIGURE 01 • An abstract attack story and places defenders can interrupt it

A cyberattack is an attempt to misuse a system, account, application, or data. An attack technique is a way that an actor may try to reach a goal. The goal might be stealing information, interrupting a service, tricking a user, or changing data.

Term	Plain meaning	Example
Threat	A possible source of harm.	A scammer who wants account access.
Vulnerability	A weakness that may be abused.	A page fails to check who is allowed to view a record.
Technique	A behaviour used to reach a goal.	Phishing, credential misuse, or unsafe input handling.
Exploit	A method that takes advantage of a weakness.	A training lab demonstrates an input-handling flaw.
Indicator	A clue left in data or systems.	A sign-in pattern that is unusual for a test account.
Impact	The harm or change that results.	A service outage or unauthorised data exposure.

SAFE BOUNDARY Learning about an attack is not permission to perform it. Never test random websites, accounts, Wi-Fi networks, or devices. Use a local lab, a synthetic dataset, or a program that explicitly authorises the exact activity.

LEVEL 01

Beginner: recognise the common patterns

Learn what each pattern means, what it may look like, and the first safe defensive response.

> YOUR GOAL Finish this level when you can explain the ideas in your own words and complete its practice without copying the answer.

1. Phishing

Pause, verify, and report an unexpected message

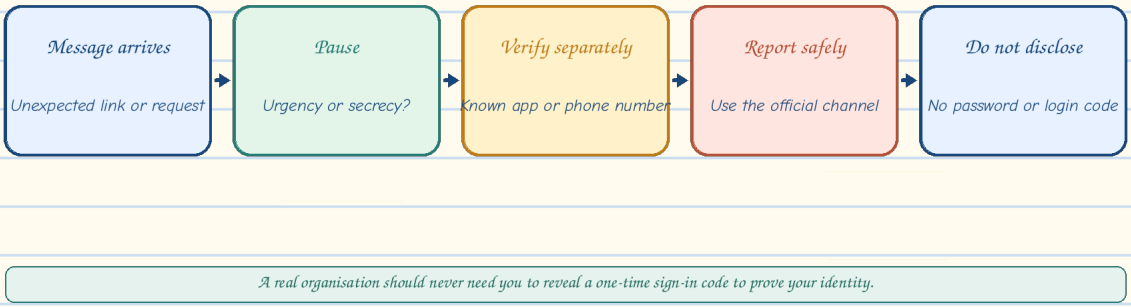


FIGURE 02 • How to verify an unexpected message safely

In plain words: A message pretends to be from a trusted person or service to push someone into clicking, opening, paying, or sharing secrets.

Fictional example: A fictional message says a student account will be disabled in ten minutes unless the student follows an unfamiliar sign-in link. The sender name looks familiar, but the reply address does not match.

What a defender might notice: Unexpected urgency, mismatched sender details, unusual links, requests for passwords or codes, unexpected attachments, or wording that does not fit the normal process.

Reduce the risk: Use MFA, password managers, mail filtering, verified contact methods, and a clear "report phishing" process. Do not enter credentials from an unexpected message.

Safe practice: Read the fictional message below and highlight two clues. Do not open anything: "From: support@example.invalid — Your account expires now. Visit [training link omitted] and enter your password."

2. Social engineering

In plain words: A person uses trust, fear, urgency, curiosity, or authority to influence someone into taking an unsafe action.

Fictional example: *A caller claims to be from IT and asks a learner to read out a one-time login code. The caller knows the school name, but the request is not part of the normal support process.*

What a defender might notice: *Pressure to keep a secret, requests to bypass normal approval, unusual payment or access requests, or demands for passwords and one-time codes.*

Reduce the risk: *Verify requests using a known official contact path. Use approval steps and train people to pause without blame.*

Safe practice: *Write a safe response: "I will not share a code. I will contact the support desk using the published contact details."*

3. Password guessing, spraying, and credential stuffing

In plain words: *These are attempts to use passwords or stolen username/password pairs to access accounts. Guessing tries possible passwords; spraying tries a small number across many accounts; credential stuffing reuses credentials stolen elsewhere.*

Fictional example: *A fictional identity log shows many failed sign-ins across several test accounts, followed by a sign-in that needs review. No real accounts are involved.*

What a defender might notice: *Repeated failures, bursts across many accounts, unfamiliar devices, impossible-looking travel, MFA prompts the user did not initiate, or sign-in patterns different from baseline.*

Reduce the risk: *Use unique passwords in a password manager, MFA or passkeys where available, rate limiting, breached-password protections, monitoring, and secure recovery procedures.*

Safe practice: *Review the supplied sample table and count failed sign-ins. Do not run a password-guessing tool or test another person's account.*

4. Malware

In plain words: *Malware is software created to perform unwanted or harmful actions, such as spying, stealing, damaging, or controlling a device.*

Fictional example: *A fictional endpoint alert reports that a suspicious attachment tried to start an unexpected process. The sample includes only the alert description, not an executable file.*

What a defender might notice: *Unexpected processes, unusual file changes, security-tool alerts, unexplained network connections, or a program starting from an unusual location. A single sign is not proof.*

Reduce the risk: *Patch systems, limit local admin rights, keep endpoint protection enabled, restrict untrusted macros and downloads, and maintain tested backups.*

Safe practice: *Examine the written alert fields in the lab worksheet. Never download or run a sample just to see what it does.*

5. Ransomware

In plain words: *Ransomware is malware or an attack pattern that blocks access to data or systems, often by encrypting files, and may include threats to leak stolen data.*

Fictional example: *A fictional organisation notices many files changing in a short time and several users cannot open shared documents. The scenario provides logs only.*

What a defender might notice: Mass file changes, unusual changes to backup services, ransom notes reported by users, endpoint detections, or sudden access problems. Validate before concluding.

Reduce the risk: Keep offline or protected backups, test recovery, patch exposed systems, limit privileges, segment critical services, and rehearse the incident process.

Safe practice: Make a recovery checklist for the fictional organisation. Do not create or execute ransomware or disable protections.

6. Denial of service (DoS / DDoS)

In plain words: A denial-of-service event tries to make a service unavailable by consuming resources or overwhelming a service. DDoS uses many sources.

Fictional example: A training dashboard shows a fictional website receiving a sudden traffic surge while response times rise and legitimate visitors have trouble connecting.

What a defender might notice: Unusual traffic volume, rising latency, resource saturation, spikes across many network sources, and provider mitigation alerts.

Reduce the risk: Use provider protections, rate limits where appropriate, capacity planning, monitoring, caching, and an incident contact path. Do not retaliate or generate attack traffic.

Safe practice: Interpret a provided chart of normal versus unusually high synthetic request counts. Do not send traffic at any real service.

7. Man-in-the-middle (MitM)

In plain words: A MitM situation happens when an attacker gets between two communicating parties and may observe or alter traffic. Correctly validated encrypted connections reduce many risks.

Fictional example: A fictional user joins an untrusted network and sees a browser certificate warning on a site they usually use.

What a defender might notice: Unexpected certificate warnings, suspicious network changes, unexpected redirects, or inconsistent connection details. These signs can also have benign causes.

Reduce the risk: Use HTTPS correctly, validate certificates, avoid sensitive work on untrusted networks when possible, use trusted VPNs where appropriate, and do not ignore warnings.

Safe practice: Explain why the browser warning should be checked instead of clicked through. Do not intercept anyone else's traffic.

8. DNS spoofing or misleading DNS results

In plain words: DNS helps translate a name into an address. If a user or resolver receives a false answer, they might reach the wrong destination.

Fictional example: A fictional lab resolver returns an unexpected address for a reserved training name, and a separate known-good record is supplied for comparison.

What a defender might notice: Unexpected DNS answers, changes outside planned maintenance, lookalike domains, unusual query patterns, or certificate/name mismatches.

Reduce the risk: Use trustworthy resolvers, protect DNS administration, monitor records, use HTTPS and certificate validation, and verify high-risk destinations through known routes.

Safe practice: Compare the two fictional DNS records and write what differs. Do not change public DNS records or redirect real users.

9. Injection flaws in web applications

A safer web-request path

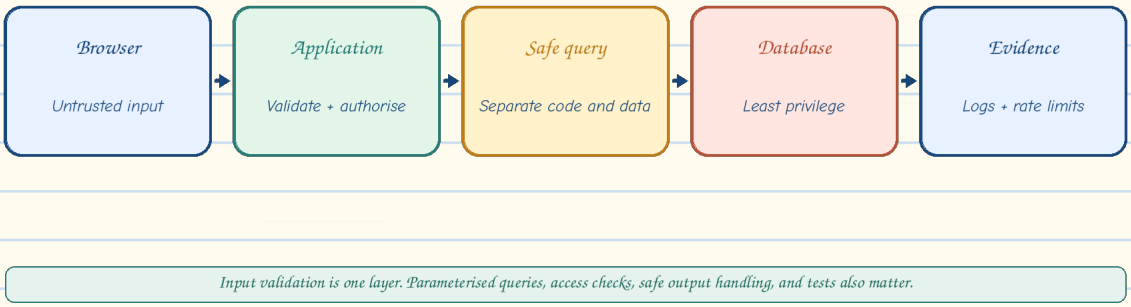


FIGURE 03 • Where web applications apply security controls

In plain words: Injection risk appears when an application mixes untrusted input with a command or query in a way that changes how the application interprets it.

Fictional example: A fictional training app builds a database query by joining raw form text into a query string instead of using parameterised queries. No live target or attack input is used.

What a defender might notice: Unexpected database errors, unusual inputs in logs, behaviour differing from normal validation, or an application security test report.

Reduce the risk: Use parameterised queries, safe APIs, server-side validation, least-privilege database accounts, and code review. Output encoding alone does not fix every injection type.

Safe practice: Compare the design idea: "join user text into query text" versus "send user text as a parameter." Explain why parameters keep data separate from query instructions.

10. Cross-site scripting (XSS)

In plain words: XSS occurs when a website handles untrusted content in a way that lets it run as script in another user's browser.

Fictional example: A fictional comment field shows untrusted text on a page without treating it safely as text. A training report explains the risk; it contains no runnable payload.

What a defender might notice: Unexpected script-like behaviour, reports from a safe test suite, risky rendering patterns, or browser-side changes associated with untrusted content.

Reduce the risk: Use context-appropriate output encoding, safe templating, input handling, and a Content Security Policy as an additional layer.

Safe practice: Review a short code-review checklist in a lab and identify the output point that should treat user content as text.

11. Broken access control

In plain words: The application does not correctly check whether a logged-in user is allowed to see or change the requested item.

Fictional example: In a fictional app, a student account can view a record that belongs to another demo account because the server checks only that a user is signed in.

What a defender might notice: A user can reach data outside their role, server responses reveal unauthorised records, or an authorisation test finds inconsistent permissions.

Reduce the risk: Check permission on the server for every sensitive action and object, deny by default, minimise access, and test owner/role boundaries.

Safe practice: Use two fictional accounts in a purpose-built training app and a provided checklist. Never try another person's real record.

12. Security misconfiguration

In plain words: A system has unsafe settings, default access, unnecessary services, overly broad permissions, or incomplete logging.

Fictional example: A fictional storage service intended for private documents is configured to allow public reads.

What a defender might notice: Unexpected public access, default settings, exposed debug pages, broad cloud permissions, missing logs, or services that do not need to be exposed.

Reduce the risk: Use secure baselines, least privilege, configuration review, patching, secret management, continuous checks, and change approval.

Safe practice: Read a fictional cloud configuration checklist and mark which settings need an owner to confirm them. Do not scan or change public systems.

13. Software supply-chain risk

In plain words: A supply-chain incident affects the software, dependency, update, build system, or service that an organisation trusts.

Fictional example: A fictional project update includes a dependency that was not reviewed. The exercise asks learners to inspect the project's dependency and release review process.

What a defender might notice: Unexpected package changes, unusual build activity, unsigned or unverified artefacts, dependency alerts, or a release that does not match the approved process.

Reduce the risk: Pin and review dependencies, use trusted sources, protect build credentials, verify artefacts when supported, monitor advisories, and keep a clear inventory.

Safe practice: Review a sample dependency-change record and list the evidence needed before approving the update.

14. Privilege escalation and lateral movement

In plain words: Privilege escalation means gaining more access than intended. Lateral movement means moving from one system or account to another within an environment.

Fictional example: A fictional endpoint timeline shows a standard account followed by an unexpected administrator-level action and access to a second lab host.

What a defender might notice: Privilege changes, new remote sessions, unusual administrative tools, new host-to-host relationships, or activity outside normal role patterns.

Reduce the risk: Patch systems, use separate admin identities, enforce least privilege, segment networks, monitor remote access, and review sensitive actions.

Safe practice: Map the synthetic timeline and identify which records need review. Do not attempt to raise privileges on a real device.

15. Data exfiltration and insider misuse

In plain words: Exfiltration is data leaving an environment. Insider misuse is harmful or policy-breaking use of legitimate access by someone inside the organisation.

Fictional example: A fictional workstation transfers an unusually large set of files to a destination that is not used by the team. The data is invented and contains no private information.

What a defender might notice: Unusual transfer volume, unfamiliar destinations, access to many records, removable-media events, or activity inconsistent with the user's duties.

Reduce the risk: Use data classification, least privilege, approved sharing, audit trails, egress monitoring proportionate to the risk, and fair policy-based investigations.

Safe practice: Classify the sample transfer as "needs review" or "explained" and write which context is missing. Do not inspect anyone's private files.

LEVEL 02

Intermediate: connect the evidence

Move from memorising names to understanding patterns across events, applications, and controls.

> YOUR GOAL Finish this level when you can explain the ideas in your own words and complete its practice without copying the answer.

16. A simple attack story - and what defenders can learn

Combine sources before deciding

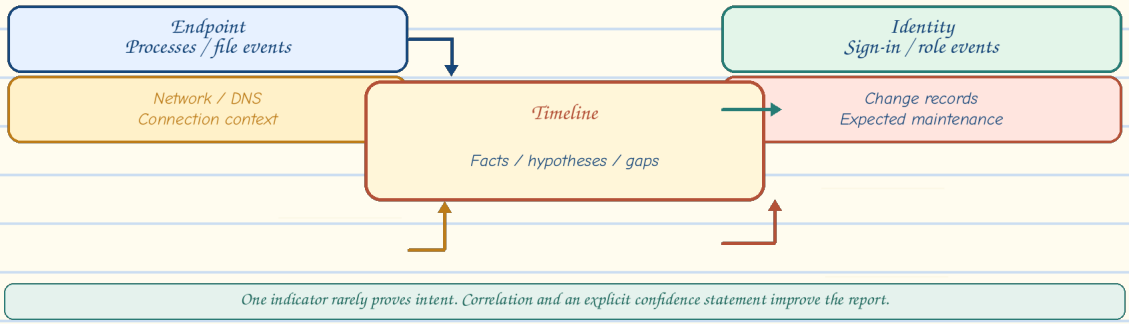


FIGURE 04 • Combine independent evidence sources before drawing conclusions

This is a fictional teaching scenario. It does not describe a real victim, and it does not include attack instructions or malware.

Possible stage	Fictional observation	Defender's question
Message received	A user reports an unexpected account notice.	Was the message expected, and was it reported through the right channel?
Link concern	The message includes an unfamiliar sign-in destination.	Can the destination be verified safely without opening it?
Identity signal	A later sign-in record differs from the baseline.	Are the identity, device, MFA result, and time trustworthy?
Application access	A sensitive application records an access attempt.	Did the server enforce the user's role and object permissions?
Response	The case owner requests identity review.	What approved containment, notification, and recovery steps apply?

> IMPORTANT A timeline is a way to organise evidence, not a claim that every stage occurred. Mark unsupported stages as unknown.

17. Read a fictional alert before deciding

> ALERT SUMMARY Rule: unusual login pattern | Account: demo.learner | Time window: 10 minutes | Results: 6 failed, then 1 success | Source data: synthetic lab CSV

What to do

1. Verify the record time, source, account, and whether the logs were collected correctly.
2. Compare with normal behaviour for this test account and similar accounts.
3. Check MFA outcome and what happened after the successful login.
4. Consider benign explanations, including a forgotten password or an approved test.
5. Write a finding with a confidence level and a list of missing context.

18. Web application risk categories

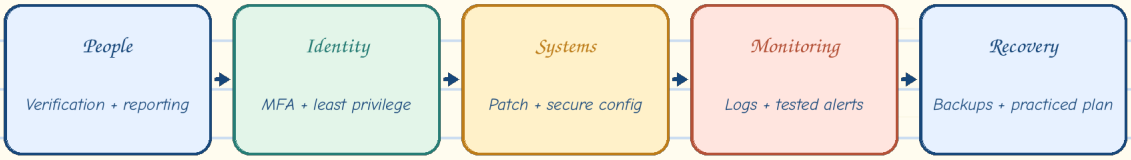
OWASP Top 10:2025 is an awareness starting point for web application security. These categories help you learn vocabulary and ask better review questions. They are not an exhaustive checklist, and their order is not a score for a specific application.

OWASP Top 10:2025 category	Beginner question
Broken Access Control	Does the server check what this user is allowed to access?
Security Misconfiguration	Are settings, permissions, debug features, and services appropriate?
Software Supply Chain Failures	Are dependencies and build/update processes reviewed and protected?
Cryptographic Failures	Is sensitive information protected appropriately in storage and transit?
Injection	Is untrusted input kept separate from commands and queries?
Insecure Design	Were security needs considered before the feature was built?
Authentication Failures	Are login, session, recovery, and MFA flows safely designed?
Software or Data Integrity Failures	Can untrusted changes reach trusted processes or updates?
Security Logging & Alerting Failures	Would important security behaviour be visible and investigated?
Mishandling of Exceptional Conditions	Do errors and edge cases fail safely?

> VERSION NOTE This section uses OWASP Top 10:2025. Check the official page before quoting categories in reports or learning material.

19. Attack lifecycle words, without the movie myths

Defensive layers around a service



Layered controls reduce reliance on any single tool, user, alert, or prevention measure.

FIGURE 05 • Layered defensive opportunities across a service

Behaviour area	What it means in plain language	Defensive source to review
Reconnaissance	Learning about a target before an action.	Scope, approved asset inventory, external exposure records.
Initial access	An attempt to enter through an account, app, or user action.	Identity, email, web, and remote-access telemetry.
Execution	A program or command starts.	Process logs, endpoint detections, parent/child process context.
Persistence	An attempt to remain after the initial access.	Approved configuration changes and startup-related records.
Privilege escalation	An attempt to obtain more permissions.	Role changes, privilege events, and patch context.
Defense evasion	An attempt to avoid or impair security visibility.	Security-control health and changes to logging.
Credential access	An attempt to obtain or misuse secrets.	Identity alerts and credential-protection telemetry.
Discovery / lateral movement	Learning about or moving between systems.	Host and account relationships, remote sessions.
Collection / exfiltration	Gathering information and potentially moving it out.	Access patterns, transfer logs, approved sharing records.
Impact	Changes to availability, integrity, or operations.	Service health, file integrity, recovery status.

LEVEL 03

Advanced: defensive analysis practice

Plan safe tests, evaluate controls, describe limitations, and turn a finding into useful action.

> YOUR GOAL Finish this level when you can explain the ideas in your own words and complete its practice without copying the answer.

20. Plan a safe attack-pattern learning exercise

Before the exercise	Write this down
Permission	Which training platform, local VM, or written scope authorises the activity?
Target	What exact lab asset is allowed, and what is explicitly out of scope?
Purpose	Which security question are you trying to answer?
Method	Use the training guide's approved steps; avoid improvising against real services.
Stop condition	Stop if the activity touches real accounts/data, disrupts a service, or scope is unclear.
Evidence	Save lab result, timestamp, assumptions, and limitations; remove private data.
Clean-up	Restore the lab state and report issues through the assigned channel.

21. Defensive control mapping

Ransomware resilience has several layers

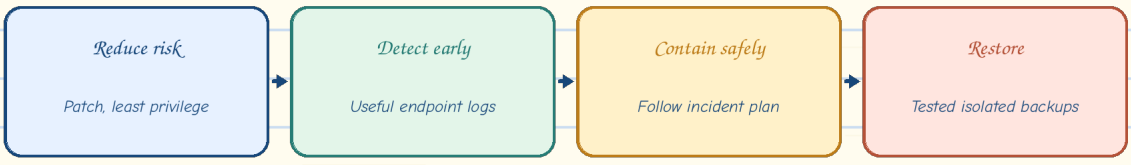


FIGURE 06 • Prevention, detection, containment, and recovery for ransomware risk

Pattern	Prevent / reduce risk	Detect / investigate
Phishing	Email filtering, MFA, verified communications, user reporting.	Message headers where appropriate, identity events, user report, safe URL reputation

Pattern	Prevent / reduce risk	Detect / investigate
		sources.
Credential misuse	Unique passwords, MFA/passkeys, throttling, recovery controls.	Failure/success sequence, device, session, MFA, baseline.
Malware / ransomware	Patching, least privilege, application controls, protected backups.	Endpoint alerts, file-change pattern, service health, recovery evidence.
DoS/DDoS	Provider mitigation, capacity planning, edge controls.	Traffic rate, latency, resource load, provider notices.
Injection / XSS	Safe query APIs, output encoding, validation, code review.	Application errors, safe test findings, relevant application logs.
Broken access control	Server-side authorisation, deny by default, least privilege.	Authorisation logs, expected role/object relationships, test reports.
Misconfiguration	Secure baseline, change review, configuration monitoring.	Drift alerts, inventory, permission and exposure checks.
Supply chain	Dependency review, protected build pipeline, verified releases.	Unexpected dependency/release change, advisory, artefact provenance.

22. Write a clear finding

A useful finding is more than a scary title. It gives the reader a fact they can verify, explains why it matters, describes uncertainty, and asks for a proportionate next step.

Field	Example wording
Title	Unusual sign-in sequence in the synthetic training dataset
Observation	Six failed sign-ins were followed by one success within the sample time window.
Evidence	Training case AUTH-04, rows 12–18; times normalised to UTC.
Meaning	The pattern deserves review; the sample does not prove compromise.
Unknowns	The source lacks verified MFA detail and the account owner's confirmation.
Recommendation	Check the permitted identity records and follow the lab escalation instructions.
Limit	Fictional data only; not a production detection-performance claim.

> QUALITY RULE Use "observed", "suggests", "consistent with", or "not established" when those phrases match the evidence. Avoid "confirmed" unless the required evidence standard was met.

Practice exercises

Exercise A — phishing: *Identify two warning signs in the fictional message from Section 1 and write one safe next step.*

Exercise B — identity signals: *Using the four-row sample from Section 17, write two observations and two unknowns.*

Exercise C — web security: *Choose broken access control, injection, or misconfiguration. Describe one secure design decision and one test question for an authorised lab.*

Exercise D — ATT&CK: *Given a fictional alert about an unusual remote session, write the defensive records you would check before mapping a behaviour.*

Exercise E — report: *Write a four-sentence finding with an observation, evidence reference, limitation, and next action.*

Answer-check guide

- *Phishing clues may include urgency, sender mismatch, unexpected login request, or an unusual destination. The safe step is to report and verify through a known channel.*
- *Observation example: "Six failures and one success are present." Unknowns might include whether MFA succeeded, who controls the account, and whether the data is complete.*
- *Broken access control: authorise each sensitive object on the server; test owner and role boundaries using two fictional accounts in the designated lab.*
- *Check the account, host, timestamp, source, expected remote administration, associated identity events, and whether the logs are trustworthy.*
- *A strong report separates what was observed from what is inferred, names the data source, states uncertainty, and recommends an approved next step.*

Quick-glossary

Use this as a lookup page. You do not need to memorise every term on the first day.

Word	Plain-language meaning
Authentication	Checking who someone or something is.
Authorisation	Checking what that identity is allowed to do.
DDoS	Distributed denial of service; service disruption from many sources.
Injection	Untrusted input changes the meaning of an instruction or query.
Lateral movement	Activity moving between systems or accounts in an environment.
MFA	Multifactor authentication; more than one kind of proof is used.
MitM	A situation where a third party sits between two communicating parties.
Persistence	An attempt to keep access or execution available over time.
Privilege escalation	Obtaining more permissions than previously allowed.
Supply chain	Software components, people, processes, and services involved in delivery.
XSS	Cross-site scripting: unsafe web handling lets untrusted content run as script.
OWASP	A community project providing web application security guidance.

Trusted references & further study

FIELD NOTE • A tool, indicator, or alert does not prove intent by itself. Start with permission and scope, collect only the evidence you are authorised to use, compare independent context, and write what is known, suspected, and still unknown. Do not test unknown techniques on real people or systems.

These notes simplify the ideas for learning. For exact definitions, implementation details, and updated guidance, return to the original publisher.

OWASP Top 10:2025 — OWASP. *A current awareness list of important web-application risk categories.*
https://owasp.org/www-project-top-ten/

OWASP Web Security Testing Guide — OWASP. *Structured web testing methodology; use only permitted testing scopes.*
https://owasp.org/www-project-web-security-testing-guide/

MITRE ATT&CK Enterprise Matrix — MITRE. *Shared vocabulary for observed adversary tactics and techniques.*
https://attack.mitre.org/matrices/enterprise/

Secure Our World — CISA. *Plain-language actions for phishing, strong passwords, password managers, and multifactor authentication.*
https://www.cisa.gov/secure-our-world

#StopRansomware Guide — CISA and partners. *Risk reduction, detection, response, and recovery for ransomware.*
https://www.cisa.gov/resources-tools/resources/stopransomware-guide

SP 800-61 Rev. 3: Incident Response Recommendations — NIST. *Risk management and incident response recommendations.*
https://csrc.nist.gov/pubs/sp/800/61/r3/final

> KEEP LEARNING A good analyst does not copy a tool output and call it truth. They record what they observed, why it matters, what else could explain it, and what should happen next.

A real lab for these notes

Topic: Recognise web attack patterns in intentionally vulnerable training applications and connect them to mitigations. These pages connect the existing OpenCyberPath notes to specific provider-hosted exercises. The card label is intentionally generic; the exact task name and direct URL are listed here so you can open the right exercise instead of a whole catalogue.

Concept checkpoints

- SQL injection occurs when untrusted input is interpreted as part of a query. Defend with parameterized queries.
- XSS occurs when untrusted content is interpreted as active page code. Defend with context-aware output encoding and safe templating.
- Access-control flaws let a user perform an action the application should deny. Enforce authorisation on the server.
- A lab exploit proves only the behaviour of that specific intentionally vulnerable instance.

Lab: SQL injection - retrieve hidden data *[PortSwigger Web Security Academy]*
https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data
Access note: Launch the intentionally vulnerable lab from the page. The exercise is contained in a provider-generated instance.

Lab: Reflected XSS into HTML context with nothing encoded *[PortSwigger Web Security Academy]*
https://portswigger.net/web-security/cross-site-scripting/reflected/lab-html-context-nothing-encoded
Access note: Launch the specific training instance from the page. Use the sample only inside that instance.

Local deliberately vulnerable training app *[OWASP Juice Shop]*
https://github.com/juice-shop/juice-shop
Access note: Open-source training app that must be run locally in an isolated environment. Do not expose it to the public internet.

ACCESS CHECK Open the exact task page and verify the current login, account tier, VM launch button, free/paid terms, and any cost notes before starting. A provider may change availability or retire a challenge.

Before you click Start

- Write down the provider's target/scope and use only the instance it creates for you.
- Keep this notebook beside the exercise. Use its original course data before trying commands on a local machine.
- Capture timestamps, filenames, row/packet IDs, and a short explanation; redact credentials, tokens, and personal data.
- If a page says retired, premium, or downloadable-only, use the alternative listed in the access note or return to the matching interactive lab above.

COMMAND NOTEBOOK - COPY ONE LINE AT A TIME

Commands you can explain

The intentionally vulnerable Juice Shop must stay bound to 127.0.0.1. Web test input belongs only in the specific academy-generated lab instance.

First download the matching sample file(s) from the existing card when a command names one. Keep the data in a practice folder and never execute strings stored in evidence.

LAUNCH LOCAL-ONLY JUICE SHOP

```
docker run --rm -p 127.0.0.1:3000:3000 bkimminich/juice-shop
```

Starts the intentionally vulnerable app on loopback only. Requires Docker and a downloaded image; stop it with Ctrl+C.

VERIFY LOCAL-ONLY APP RESPONDS

```
curl -I http://127.0.0.1:3000/
```

Makes a request to the local app you just launched, not a remote site.

INSPECT THE GENERATED ACADEMY LAB

```
curl -i "https://YOUR-LAB-ID.web-security-academy.net/"
```

Replace placeholder only with the generated PortSwigger lab hostname. Use the assigned lab's own instructions for the exercise.

SAFE XSS PROOF INSIDE DESIGNATED LAB

```
Enter the academy page's provided beginner example in the lab search field, then observe whether
it is encoded or interpreted.
```

Use the exact training page linked above and follow its exercise steps. Do not copy test strings to real websites.

SOURCE REPOSITORY CLONE

```
git clone --depth 1 https://github.com/juice-shop/juice-shop.git
```

Alternative to the container image; it retrieves source code locally and does not attack another system.

COMMAND HABIT Check the current working directory, shell, target, and expected effect before pressing Enter. Save the exact command and the output that mattered.

RUN THE EXERCISE - RECORD WHAT THE EVIDENCE REALLY SHOWS

Guided lab run

- 1. Choose one exact lab page; wait until the provider generates the isolated instance.
- 2. Read the objective and permitted actions before submitting any test input.
- 3. Reproduce the smallest input that demonstrates the assigned flaw in that instance only.
- 4. Record actual/expected behaviour, a minimal redacted request, and a defensive remediation.
- 5. Stop the instance when done; do not reuse the payload on unrelated services.

Evidence checklist

- [ ] What exact lab instance or sample file was used?
- [ ] Which command/query and time range produced the observation?
- [ ] What is the directly observed fact, separate from your inference?
- [ ] What benign/alternative explanation could fit?
- [ ] What extra record or control would help confirm the conclusion?

Check your understanding

Q Why can the same test string behave differently on a real site?

Think it through: Input handling, filters, authentication, and application context differ; never infer permission or vulnerability from another lab.

Q What is a correct mitigation for SQL injection?

Think it through: Use parameterized queries or prepared statements and least-privilege database access.

Q What is a correct mitigation for XSS?

Think it through: Use context-aware output encoding, safe templating, and avoid unsafe HTML sinks.

NOTEBOOK CLOSE-OUT Finish with: Observation / Evidence source / Why it matters / Alternative explanation / Next check / Limitation. A careful "not enough evidence yet" is a valid professional result.